

input, while our game loop looked something like this:

```
bool bGameRunning; // variable to hold a boolean if the game is running or not.

int main() {
    InitWindow(); // we create and initialize a window
    InitGraphics(); // we initialize the graphics interface
    InitSound(); // inits sound interface with the sound hardware
    InitInput(); // initialize input
    InitPhysics(); // initialize physics
    InitNetwork(); // initialize network
    bGameRunning = startGame(); // starts game returns true if all went well

    while(bGameRunning) { // this is our loop, while the game is running keep
        looping
        updateInput(); // updates the Input
        checkCollision(); // check for collision player position + world + NPC
        updateAI(); // update all AI/NPC in game
        updatePlayer(); // update player according to input
        updatePhysics(); // update Physics of the game world
        updateServer(); // send messages to server + other players
        updateGame(); // update game, see it has ended or not
        updateDisplay(); // update screen according to all changes
    }

    return 0; // game ended, program flow exits the loop and we exit the program.
}
```

A constantly updating chain of commands keeps the game world and its creatures alive even when there is no update from the user. As you can imagine, more functions and functionality would be added to the list for handling things like networking, audio, sound, music, more input devices, physics, etc. As this system grows, you should bring your object-oriented programming skills to the table to correctly classify the objects. So you may have an idea by now of what game development might involve. There's a heck of a lot more, however!

What tools and technology you use largely depends on how you want to go on developing the game. Game development involves a whole lot of tools and technologies—from your content-creation pipeline to developing tools for your game and, finally, to the actual game itself.

A great deal of assets for the game are created using software like Autodesk's Maya and 3D Max. Development usually takes place using very expensive IDEs, and management software like Microsoft's Visual Studio and the likes. If you are not making middleware (software and technologies that facilitate the job of a middle man in game development—which could range from file formats, 3D graphics, physics, network engines, to whole game engines), then you would be using them and they don't come cheap. Apart from indie licences of some game engines (Torque, to name one), almost all middleware is expensive and comes with very restrictive licences.

Enter, free and open source software!

You can save a lot of time and money by carefully selecting a set of FOSS tools in your game development lifecycle. Apart from saving time and money, FOSS tools and technologies have been time-tested and moulded into what they are by strong communities of developers, as well. There are a plethora of tools and technologies out there to help you get started and get along. So let's look at some of them.

Graphics engines and general-purpose libraries

Graphics engines are libraries that interface with your graphics hardware and draw everything you see in your game. You can start writing this component from scratch, focusing on proprietary DirectX or free OpenGL—both industry standard interfaces to graphics hardware. Writing one would involve selecting an interface; what scene graph algorithms to use; how to load and manage 3D data, and how to send it to the graphics hardware; creating the means to set up light; working with textures; animation; IK; and a whole lot more. You can save a lot of trouble and jump into developing your game rather than spending time deciding how you do translations to matrices, by selecting a ready-made engine that does all this for you, out-of-the-box.

There are many free and open graphics engines out there—OGRE, Irrlicht, Crystal Space, Panda3D and the Blender Game Engine are some that stand out from the rest. Blender is primarily a 3D modelling and animation software. It can also do post-production work like editing and compositing. Apart from serving your content-creation pipeline, it has a powerful game engine as well. Some of these graphics engines sometimes provide you with not only graphics, but also other functionality like the GUI, input, post effects, file management, 3D functions, mathematics, etc.

Panda3D from CMU and Disney gives you a whole lot of tools that you can use for all your game development needs. Some of them are code for the scene graph and for rendering, while some are for graphic asset loading, timers, tasks and event handling, sound, input, video, the GUI, post FX, procedural content generation, collisions and physics. These tools also include code for mathematics, finite state machines, scripting, networking, a huge set of sample programs and code to start from, packaging and distribution solutions, debugging and performance tuning, and tools for third-party software.

The list of libraries can go on and each has its pros and cons. Testing and selecting the libraries that you require solely depends on the kind of game you are developing and the requirements you have. There are a lot of general-purpose libraries like the infamous SDL, ClanLib, Allegro and SFML that do a lot more than talk with your graphics hardware, providing file system functions, file management, mathematics, 3D functions, GUI, input and networking, to name a few. There are more libraries like the OpenSceneGraph, OpenGLut and FreeGlut that deliver smaller frameworks for your 3D application needs.